



Predicting Flaky Test in Cross-Project Scenario

KELOMPOK A01

NRP-Mahasiswa

6025261018 - Firmansyah Achmad Adam

6025261006 - Bima Jati Kusuma

6025261011 - Ali Mangsur

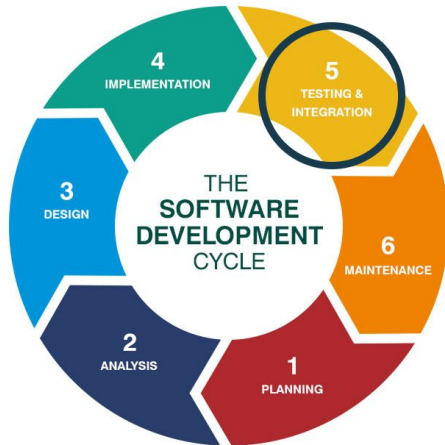
6025261019 - Faradiba Annisa

6025261021 - Ida Bagus Ari Widhiana



What is Flaky Test?

Flaky test adalah *test* perangkat lunak menggunakan source code yang sama persis namun hasilnya tidak konsisten, misal kadang-kadang *failed* padahal sebagian besar hasilnya *passed*.



Tahapan SDLC

Dalam konteks Software Development Life Cycle (SDLC), topik Flaky Test termasuk ke dalam tahapan **Testing**.

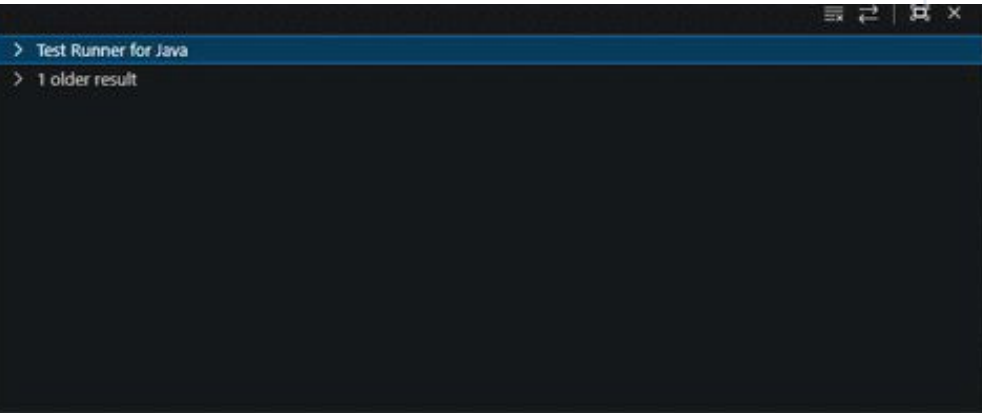
Tujuan & Manfaat

Dengan mengetahui Flaky Test pada tahapan Testing akan membantu developer dalam mencegah terjadinya masalah pada perangkat lunak saat sebelum masuk ke lingkup production.



Example of Flaky Test

```
1 class FlakyExampleTest {
2     private int counter = 0;
3
4     @RepeatedTest(100)
5     void testAsyncOperationTimeout() throws InterruptedException {
6         // Reset counter untuk setiap perulangan
7         counter = 0;
8         CountDownLatch completed = new CountDownLatch(1);
9
10        // Mensimulasikan proses asinkron (misal: background job, request API, webhook)
11        Thread backgroundTask = new Thread() -> {
12            try {
13                // simulasi latensi jaringan atau beban server.
14                Thread.sleep((long) (Math.random() * 105));
15                counter = 1;
16            } catch (InterruptedException e) {
17                Thread.currentThread().interrupt();
18            } finally {
19                completed.countDown();
20            }
21        };
22
23        backgroundTask.start();
24        // Menunggu sampai task selesai.
25        boolean finished = completed.await(100, TimeUnit.MILLISECONDS);
26        assertTrue("Background task tidak selesai dalam batas waktu (Flaky Test)", finished);
27        assertEquals(1, counter, "Counter seharusnya sudah diupdate menjadi 1");
28    }
29 }
```





Literature Review

A01 - Predicting Flaky Test in Cross-Project Scenario

Judul & Tahun	Ringkasan Penelitian	Metode/ Algoritma	Metrics & Hasil
FlakeFlagger: Predicting Flakiness Without Rerunning Tests (2021)	- Menyusun dataset <i>flaky test</i> bernama FlakeFlagger berdasarkan hasil 10.000x menjalankan setiap <i>test</i> dari 24 proyek. - Menguji seberapa efektif dataset FlakeFlagger dalam memprediksi <i>flaky test</i>. Pengujian menggunakan <i>classifier</i> berbasis algoritma <i>supervised learning</i>.	(Untuk bagian <i>classifier</i>) Data sampling: SMOTE & random undersampling Algoritma klasifikasi: Decisions Tree (DT), Random Forest (RF), Support Vector Machine (SVM), Multilayer Perceptron (MLP), Naive Bayes (NB), Adaboosting (Ada), and K-Nearest Neighbor (KNN)	(Metrics hanya untuk Random Forest) Precision 60%, recall 74%, F1 66%, AUC 86%
Static test flakiness prediction: How Far Can We Go? (2022)	- Menggunakan pendekatan <i>static metrics</i> (nilai yang bisa didapatkan dari membaca <i>code</i> tanpa menjalankannya). - Melakukan klasifikasi dari <i>static metrics</i> yang sudah dihitung.	Code metrics: CBO, LOC, McCabe, Halstead, LCOM2/5, WMC, RFC, MPC, TLOC Data sampling: under-sampling (NearMiss), random undersampling, over-sampling (SMOTE and its advanced versions) Algoritma klasifikasi: Random Forest (+ VIF dan Information Gain) Optimasi hyper-parameter: Random search	F-measure (untuk kombinasi VIF + Information Gain + SMOTE + Random Forest): 70%
A Large-Scale Empirical Investigation Into Cross-Project Flaky Test Prediction (2024)	Menguji model prediksi <i>flaky test</i> pada skenario <i>cross-project</i> dengan 27 kombinasi pipeline.	Feature Scaling: No scaling, min-max, Z-score Feature Selection: No selection, VIF + Information Gain, PCA Data sampling: No balancing, SMOTE, random undersampling Algoritma klasifikasi: Random Forest Validasi: leave-one-out cross-validation (per proyek) Filtering: Transfer Component Analysis (TCA), IG_SM_FS_TCA, Burak Filter, TrAdaBoost	Kombinasi pipeline & filtering terbaik, yaitu PCA + Random Undersampling + Random Forest dan TrAdaBoost_75, menghasilkan rata-rata precision 0.81, recall 0.63, AUC-PR 0.81, dan F1 0.7.



Dataset Overview

Dataset berasal dari hasil penelitian FlakeFlagger : <https://zenodo.org/records/445072>

Test Result : File ini adalah inti dari temuan empiris FlakeFlagger: mencatat riwayat keandalan setiap method pengujian setelah dieksekusi 10.000 kali. Memiliki 7 fitur dan 22245 baris data

Project => Nama proyek pengujian (contoh: activiti-activiti, apache-commons-exec).

Test => Nama lengkap pengujian (Package.ClassName#testMethodName).

IsFlaky => Label Utama (Ground Truth). Bernilai 1 jika pengujian terbukti flaky, 0 jika konsisten (selalu pass/fail).

NumFailingRuns => Berapa kali pengujian ini gagal dari total 10.000 eksekusi.

NumPassingRuns => Berapa kali pengujian ini berhasil lulus dari total 10.000 eksekusi.

FirstFailingRunID => Nomor run (eksekusi ke berapa) di mana pengujian pertama kali gagal (-1 jika tidak pernah gagal).

FirstPassingRunID => Nomor run di mana pengujian pertama kali lulus (-1 jika selalu gagal).

UniqueFailingExceptionTypes => Jumlah tipe exception / error berbeda yang memicu kegagalan.



Dataset Overview

A01 - Predicting Flaky Test in Cross-Project Scenario

Test Features : File ini berisi kumpulan metrik dan fitur yang diekstraksi dari setiap test case. Memiliki 29 fitur dan 22236 baris data.

Identitas Pengujian

- Unnamed: 0
- Test_name
- Project
- testClassName
- testMethodName
- flaky

Test Smells

- Assertion-roulette
- Conditional-test-logic
- Eager-test
- Fire-and-forget
- Indirect-testing
- Mystery-guest
- Resource-optimism
- Test-run-war

Metrik Kode & Eksekusi

- testLength
- numAsserts
- numCoveredLines
- ExecutionTime
- projectSourceLinesCovered
- projectSourceClassesCovered
- num_third_party_libs

Riwayat Evolusi Kode

- hIndexModificationsPerCoveredLine_window5 (5 commit terakhir)
-
- hIndexModificationsPerCoveredLine_window10000 (hingga 10.000 commit terakhir)



Plan

A01 - Predicting Flaky Test in Cross-Project Scenario

Data cleaning:

- Menghapus baris yang punya missing value
- Menghapus seluruh project yang punya *missing value*

Data scaling:

- Standardization
- Normalization
- Robust scaler

Pembagian data latih dan data uji:

- *Leave one project out* (1 proyek untuk data uji dan sisanya sebagai data latih)

Penanganan data tidak seimbang:

- SMOTE (oversampling)
- Random undersampling (undersampling)
- Class weight
- Tanpa penanganan apa-apa

Algoritma klasifikasi:

- XGBoost
- Random Forest



Pre Eliminary Experiment

A01 - Predicting Flaky Test in Cross-Project Scenario

Tahap Pengujian Awal : Analisis awal dan validasi skenario pengujian sebelum eksekusi eksperimen utama.

01. Setup & Lingkungan

- Konfigurasi dataset awal
- Pemeriksaan kelengkapan fitur
- Validasi dependensi lingkungan

02. Uji Coba Model

- Baseline model testing
- Evaluasi metrik awal
- Deteksi awal bias data

03. Analisis & Temuan

- Pencatatan hasil sementara
- Identifikasi kendala teknis
- Rekomendasi iterasi berikutnya



Terima Kasih

ANY DISCUSSION?
